

SPRINT 6 COMPLETION REPORT

SHADOW OS v2 — Knowledge Manager Foundation

' COMPLETE — 27/27 TESTS PASS

Date: 2026-07-12 · Sprint 6 · FOREX ENGINE PRO

Executive Summary

Sprint 6 builds the **Knowledge Layer**

: a read-only layer that organizes the

measured Shadow LAB research (the Sprint 5 tables) into **versioned**, immutable, content-addressed, fully-provenanced knowledge artifacts. It answers the next question after **"what is the system learning?"** — namely **"what does the system now know, and how confident is it?"** — while **never** touching a single live, shadow, or risk decision.

The layer consumes **only**

the shadow_* research tables (plus the append-only

events stream they derive from) and writes **only**

its own knowledge_*

tables. It is **contract-first**, additive, append-first, idempotent, and

reversible, **gated behind a new flag KNOWLEDGE_LAYER that defaults off**, in

which state the builder never starts and the system behaves exactly as before.

Every artifact is **content-addressed**

: its checksum is computed over the built

content only, never over provenance. This is the load-bearing durability

property — a restart or redeploy mints a fresh run_id (and possibly a new

build_id/config_hash), yet rebuilding the same research produces the **same**

checksum **and therefore**

no new version. Knowledge accumulates; it never

churns on restart. Each artifact still carries the full provenance triple

(run_id, build_id, config_hash) in dedicated columns for auditability.

The production entrypoint index.js — which holds the live Engine A/B/C/D

decisions — was **not modified**

(its git diff is empty). No DROP, DELETE, or

TRUNCATE exists anywhere in the migration or manager code.

Verdict: **COMPLETE** — 27/27 Sprint 6 tests passing; index.js provably

untouched; flag-off proven to be a complete no-op; restart proven not to churn

versions.

Objective & Binding Constraints

Deliver a knowledge layer over the measured Shadow LAB research with **zero risk**

to live trading, under a binding constitution:

- **Knowledge NEVER influences** live / shadow / risk decisions. It is a pure read-of-research, write-of-knowledge layer.

- **Consume ONLY** the shadow_* research tables + events + existing Postgres.
- **index is stays FROZEN**: idempotent / reversible / contract-first
~~NO DROP / DELETE / TRUNCATE anywhere. All DDL is IF NOT EXISTS /~~
 ADD COLUMN IF NOT EXISTS. (Test teardown deletes only its own namespaced seed rows, mirroring the Sprint 5 suites.)
- **Flag-gated**: KNOWLEDGE_LAYER off = a complete no-op.
- **Content-addressed & provenanced**: every artifact carries run_id + build_id + config_hash; the checksum is over **content only** so restarts never churn versions.
- Each phase green before the next.

What Changed

The work was delivered in seven tasks (T001–T007), each gated on green tests.

1. Schema — telemetry/migrations/006_knowledge_foundation.sql (T001)

Purely additive DDL over the knowledge_artifacts table (defined back in migration 001, zero rows before this producer existed) plus one new table:

- **knowledge_artifacts** — extended with nullable provenance + source-window columns (run_id, build_id, config_hash, source_window_from, source_window_to) via ADD COLUMN IF NOT EXISTS. Provenance is kept in **real columns, not inside value**, so value stays pure content and the content-only checksum is never polluted by a new boot's run_id. Indexes added on config_hash, domain, and created_at.
- **knowledge_snapshots** — a new append-first **manifest** time series: one row per distinct active knowledge set, keyed dedupe_key = manifest_checksum (UNIQUE), capturing artifact_count, total_bytes, the full manifest JSONB, and the provenance triple. Never mutated, never deleted.

Registered in telemetry/migrations/autoMigrate.js so it auto-applies idempotently on startup (pg simple-protocol, no psql dependency), and covered by knowledgeMigration.test.js.

2. Provenance & content addressing — telemetry/managers/knowledgeProvenance.js (T002)

- **checksumValue(content)** — SHA-256 over a canonical, recursively key-sorted JSON of the artifact content. **Content-only** — provenance is excluded by construction, so two builds of identical research always match.

- **canonicalJson** — recursive key sort (arrays preserve order) for stable, insertion-order-independent hashing.
- **confidenceScore(n)** — a bounded, monotonic [0,1] score over sample size.
- **confidenceLevel** — re-exported LOW/MEDIUM/HIGH tiers (shared with Sprint 5).
- **createProvenance({...})** — self-provisions the `{ runId, buildId, configHash }` triple; **provenanceNote()** renders a compact audit string.

3. Repository — telemetry/managers/KnowledgeRepository.js (T003)

The immutable, versioned store — the only writer of the `knowledge_*` tables:

- **upsertVersion(domain, artifact, content, prov, meta)** — a ****compare-and-set upsert****: hashes the content; if the active artifact's checksum matches, it is a **no-op** (no new version); if it differs (or none exists), it inserts a new version, supersedes the previous active row (`superseded_at`), and stamps `migration_from` at the prior version's id — all inside a single PG transaction.
- **insertSnapshot(manifestRows, prov)** — records a manifest snapshot keyed by `manifest_checksum`; an unchanged active set **dedupes** (idempotent restart).
- **Read APIs**: `getActive` / `getVersion` / `getHistory` / `listActive` / `exportActive` / `statistics` / `listSnapshots` — all pure reads.
- A sequential (`_upsertSeq`) fallback mirrors the PG CAS path for the SQLite adapter used in `local/dev`.

4. Manager — telemetry/managers/KnowledgeManager.js (T004)

The orchestrator that turns research into the seven knowledge artifacts, each a pure SQL aggregation over the `shadow_*` tables:

Domain / Artifact	Built from
expectancy / history	<code>shadow_expectancy_snapshots</code> — expectancy time series per scope
engines / statistics	<code>shadow_engine_evals</code> — per-engine behaviour + winrate (abstention-aware)
patterns / validated	<code>shadow_signals</code> + <code>shadow_outcomes</code> — validated pattern buckets
market / fingerprints	<code>shadow_signals</code> + <code>shadow_outcomes</code> — per-fingerprint outcomes
config / history	<code>shadow_* config_hashes</code> observed + current env surface
confidence / history	<code>shadow_expectancy_snapshots</code> — confidence tiers over time
experiments / metadata	<code>shadow_* run/build/config</code> experiment provenance

- **snapshotAll()** builds all seven, CAS-upserts each, then records the manifest snapshot; returns `{ ok, changed, results, snapshot }`.
- **Lifecycle**: `start()` (an unref'd 15-minute poll — never keeps the process alive) / `stop()`; construction is **side-effect-free** (no timer, no writes).

- **Read APIs:** getStatistics / getArtifact / listArtifacts / listSnapshots / exportAll.
- **Null-safety helpers** numOrNull / intOr0 / rate respect the Number(null) === 0 trap — an abstaining engine's "no winrate" stays null, never a fabricated 0. **tolso** normalizes timestamps so checksums are stable across driver representations.
- A domainPrefix option namespaces artifact identities for isolated testing.

Load-bearing invariant (locked): a builder's content must **never** depend on

this.provenance (run_id/build_id/config_hash). The checksum is content-only; any provenance leak into content would churn versions on every restart. Two builders were corrected during T005 to honour this (see below).

5. Wiring — flag-gated (T006)

- Barrel export of KnowledgeManager + KnowledgeRepository from telemetry/managers/index.js.
- telemetry/server.js: a flag KNOWLEDGE_LAYER (default **off**) gates the builder start inside app.listen. **Five read-only, additive** endpoints —
 /api/knowledge/status, /api/knowledge/artifacts,
 /api/knowledge/artifacts/:domain/:artifact (supports ?version= / ?history=1),
 /api/knowledge/snapshots, /api/knowledge/export — are always registered and report knowledgeEnabled. When the flag is off, the builder never starts (zero behavior change); the manager instance is side-effect-free until start().

6. Verification (T005)

The full Sprint 6 suite runs green against the development PostgreSQL. Two real bugs were found and fixed during verification (both content-only-checksum violations — see Design Notes). No live-trading code was touched.

7. Deliverables (T007)

This report (+ PDF), CHANGELOG.md, replit.md, and docs/architecture/MASTER_ARCHITECTURE.md updates, memory notes, and an architect review.

Requirements Compliance

#	Requirement	Status
---	-------------	--------

1	Knowledge NEVER influences live/shadow/risk	' (read-of-research, write-of-knowledge only; no feedback path)
2	Consume ONLY shadow_* + events + existing PG	' (all seven builders read shadow_*; no other source)
3	index.js FROZEN	' (diff empty)
4	Additive / append-first / idempotent / reversible	' (all DDL IF NOT EXISTS; CAS upsert + manifest dedupe)
5	Flag KNOWLEDGE_LAYER off = no-op	' (builder never starts; construction side-effect-free — tested)
6	No DROP / DELETE / TRUNCATE	' (none in migration/manager; test deletes only own seed rows)
7	Every artifact carries run_id+build_id+config_hash	' (stamped on every insert, in real columns)
8	Content-addressed; restart mints no new version	' (content-only checksum; proven by the recovery suite)
9	Versioned + immutable (supersede, never mutate)	' (CAS upsert; superseded_at + migration_from chain)
10	Each task green before the next	' (T001!'T007)

Verification

Full suite run against the development PostgreSQL

(node --test --test-reporter=spec --test-concurrency=1):

```
' knowledgeProvenance — content-only checksum / determinism / tiers (7 tests)
' knowledgeMigration — 006 registered + applied + idempotent (7 tests)
' knowledgeManager — build/version/idempotency/supersede/export (7 tests)
' knowledgeRecovery — restart mints no new versions (3 tests)
' knowledgeFeatureFlag — flag-off no-op; writes only on start (3 tests)
!9 tests 27 !9 pass 27 !9 fail 0
```

Highlights proven by the suite:

- **Content addressing** — checksumValue is deterministic, key-order independent, and **excludes provenance** (identical content != identical hash).
- **Build correctness** — an end-to-end seed (`events != ShadowLabManager != shadow\_\* != KnowledgeManager`) produces all seven artifacts; built content reflects the seeded fingerprint, expectancy scope, and training-event counts.
- **Idempotency** — re-running snapshotAll() over unchanged research mints **0** new versions and the manifest snapshot dedupes.
- **Versioning** — a new resolved signal supersedes the affected artifact to v2 with migration_from pointing at v1 and exactly one active row.
- **Recovery / rollback-safety** — a second manager with a **different provenance** (new run_id/build_id/config_hash) rebuilding the same research mints **0** new versions and the active rows **retain the original provenance** — proving a restart never churns accumulated knowledge.
- **Flag-off no-op** — constructing a KnowledgeManager installs **no timer**

is **not running**
 , and writes
nothing
 ; read-only calls never write; writes
 happen **only**
 on an explicit start()/snapshotAll().
 Zero-live-change proven by git diff: index.js shows 0
 changed lines; the
 only touched files are the new knowledge modules/tests, the barrel export, the
 additive server.js wiring, the additive migration, and documentation.

Design Notes & Rationale

- **Why content-only checksums (the core decision):** the artifact identity is its **content**
 , not the process that built it. Provenance (run_id/build_id/
 config_hash) lives in dedicated columns, never inside value. This is what
 makes the layer rollback-safe: a redeploy re-derives the same knowledge and
 recognizes it as unchanged. Any provenance leak into content silently churns a
 new version on every boot — so it is treated as a hard invariant.
- **Two bugs fixed during verification, both invariant violations:**
 1. Config history embedded the manager's own config_hash and an isCurrent
 flag in its content, coupling content to provenance. Fixed by dropping those
 fields; the "current config" is now captured via the **env-derived surface**
 (which changes only on a real config change) plus the observed config history
 — both provenance-independent.
 2. The integration test wrongly assumed adding a resolved signal leaves the
 expectancy artifact untouched. In fact ShadowLabManager.reconcileAll()
 appends an expectancy snapshot whenever trades resolve, so expectancy
 legitimately changes too. The fragile assertion was removed; idempotency of
 unchanged content is already proven by a dedicated test.
- **Why the knowledge layer reads only research, never index.js:** the live
 entrypoint is FROZEN and the layer must be reversible and risk-free. Deriving
 knowledge purely from the append-only research tables means it can be turned on,
 off, or rebuilt from scratch with no coupling to — and no risk to — live trading.
- **Why a CAS upsert + manifest snapshot:** the CAS upsert guarantees an unchanged
 build is a true no-op (no version churn), while the manifest snapshot gives an
 append-first, dedupe-keyed audit trail of exactly what the active knowledge set
 was at each point in time.

Residual Notes (non-blocking)

1. First real population happens on the Railway boot with the flag on.

Enabling KNOWLEDGE_LAYER=on (with SHADOW_LAB_RESEARCH=on populating the research tables) begins live knowledge building; the operator can confirm via `/api/knowledge/status` and `/api/knowledge/artifacts`.

2. Knowledge quality tracks research volume.

Confidence tiers are LOW until the research tables accumulate enough resolved trades (`VALIDATION_MIN_SAMPLE = 30`). Early artifacts are honest about their low confidence rather than fabricating it.

3. Builders cap at MAX_ROWS = 500 per aggregation

to bound artifact size;
this is a foundation-sprint default and can be raised without schema change.

4. The manifest snapshot captures the global active set.

In a multi-tenant or prefixed scenario the manifest spans all active artifacts by design; tests use a unique domainPrefix to isolate their own identities.

Operator Runbook — Enabling the Knowledge Layer

1. Ensure production PostgreSQL is attached (Sprint 4.1). On boot, migration 006 auto-applies idempotently.
2. Ensure the research tables are being populated: set SHADOW_LAB_RESEARCH=on (Sprint 5).
3. Set KNOWLEDGE_LAYER=on in the Railway service variables and redeploy.
4. On boot the log shows
[SERVER] KNOWLEDGE_LAYER=on — starting knowledge builder (read-only).
5. Inspect via:
 - GET `/api/knowledge/status` — store statistics, last build, provenance.
 - GET `/api/knowledge/artifacts` — the active knowledge set (metadata).
 - GET `/api/knowledge/artifacts/market/fingerprints` — one artifact (add `?history=1` for its version chain, `?version=N` for a specific version).
 - GET `/api/knowledge/snapshots` — the manifest audit trail.
 - GET `/api/knowledge/export` — the full read-only knowledge bundle.
6. To fully revert to pre-Sprint-6 behaviour, unset the flag (or set it to off) and redeploy — the builder stops; the knowledge tables remain (append-first).

Files Touched

File

Change

telemetry/migrations/006_knowledge_foundation.sql	New — provenance columns on knowledge_artifacts + new knowledge_snapshots (all IF NOT EXISTS)
telemetry/migrations/autoMigrate.js	Register 006 in the migration set
telemetry/managers/knowledgeProvenance.js	New — content-only checksum, canonical JSON, confidence score, provenance
telemetry/managers/KnowledgeRepository.js	New — immutable versioned store (CAS upsert + manifest snapshot + read APIs)
telemetry/managers/KnowledgeManager.js	New — seven SQL-aggregation builders + snapshotAll/lifecycle/read APIs
telemetry/managers/index.js	Barrel-export the knowledge layer
telemetry/server.js	Flag-gated (KNOWLEDGE_LAYER, default off) start + 5 read-only endpoints
telemetry/tests/unit/knowledgeProvenance.test.js	New — content-only checksum / determinism / tiers (7 tests)
telemetry/tests/integration/knowledgeMigration.test.js	New — 006 registered + applied + idempotent (7 tests)
telemetry/tests/integration/knowledgeManager.test.js	New — build/version/idempotency/supersede/export (7 tests)
telemetry/tests/integration/knowledgeRecovery.test.js	New — restart mints no new versions (3 tests)
telemetry/tests/integration/knowledgeFeatureFlag.test.js	New — flag-off no-op; writes only on start (3 tests)
docs/reports/SPRINT_6_REPORT.md (+ .pdf)	This report
CHANGELOG.md, replit.md, docs/architecture/MASTER_ARCHITECTURE.md	Documentation updates

Untouched (per constraints): index.js.

FOREX ENGINE PRO · SHADOW OS v2 Migration · Sprint 6 · 2026-07-12